

DDR3 → DDR5 Component Reuse Assessment

Source project: DDR3 DRAM Controller (Axiado Corporation, ASIC implementation)

Purpose: Identify which existing DDR3 controller RTL, verification infrastructure, and architectural patterns can carry over to a DDR5 controller, and flag what must be rewritten or built new.

How to use this document

Every module from the DDR3 project is assigned one of five reuse tiers. Use this to scope work packages — Tier 1/2 items can be assigned early and in parallel with architecture decisions; Tier 4/5 items need a DDR5 protocol spec review before any RTL is written.

Tier	Meaning
T1 — Direct reuse	Protocol-agnostic. Port with no logic changes, at most parameter/width edits.
T2 — Reuse with parameter updates	Same logic, but constants/widths/field definitions must be redefined for DDR5 values.
T3 — Architectural reuse, rewrite internals	The state-machine shape or design pattern is worth keeping as a template, but the content inside it is DDR3-specific and must be substantially rewritten.
T4 — Full redesign required	DDR5 changes the underlying mechanism enough that the DDR3 file is not a useful starting point beyond reference/comparison.
T5 — New component, no DDR3 analog	DDR5 introduces a requirement that does not exist in the DDR3 design at all.

1. System-side interface (CPU-facing)

This is the layer between the CPU/SoC and the DRAM-protocol-specific controller. It was designed as a decoupling layer and does not encode anything about DDR3 specifically — this is the strongest reuse case in the whole project.

File	Role in DDR3 design	Tier	Notes for DDR5
<code>memory_controller_top.sv</code>	System bus, I2C/SPI infra, top-level integration	T3	Keep the overall shape; the DDR3MC instance, SPDC interface, and DIMM register outputs need to point at new DDR5 equivalents.
<code>memory_controller_interface_to_ddr3mc.v</code>	Bridges decoupled ready/valid bus to internal command protocol; scrambles/descrambles data	T2	Bridge pattern is reusable as-is; only the target internal interface name/fields change. Scrambler call sites unchanged.
<code>RRArbiter.v</code>	Round-robin arbitration between 2 CPU ports	T1	No protocol dependency at all.
<code>Queue.v</code> (response ordering)	Tracks port ownership for in-order responses	T1	No protocol dependency at all.
<code>ddr3mc_queues.v</code>	CMDQ / WDQ / RDQ instantiation	T2	Pattern is reusable directly; WDQ/RDQ width ratios must change because DDR5 burst length is BL16 instead

File	Role in DDR3 design	Tier	Notes for DDR5
			of BL8 (PISO/SIPO conversion ratio doubles).
ddr3mc_fifo_sync.v ddr3mc_fifo_sync_piso.v ddr3mc_fifo_sync_sipo.v ddr3mc_sram.v	Generic FIFO primitives underlying the queues	T1	Fully parameterized, no protocol awareness.
scrambler.v	512-bit LFSR data scrambler	T1	Width-agnostic; works on any data bus width.
pipeline.v	Configurable pipeline register	T1	Generic utility.
reset_sync.v	Two-flop async reset synchronizer	T1	Generic utility.

2. Controller core — sequencing, configuration, command generation.

This is where DDR3-specific protocol knowledge starts to appear. Some pieces (the orchestration shape) are good templates; others (actual command encoding) must be rebuilt because DDR5 replaced discrete command pins with a multiplexed command/address bus.

File	Role in DDR3 design	Tier	Notes for DDR5
<code>ddr3mc_top.v</code>	Top-level instantiation and wiring of all sub-engines	T3	Keep as an integration template; the sub-module list itself changes (new CA-bus driver replaces control-lanes driver, possible per-sub-channel duplication).
<code>ddr3mc_imux.v</code>	Combinational mux selecting init/cal/proc command source	T2	The select pattern (<code>{init_en, cal_en, proc_en}</code> priority mux) is directly reusable; only the signal set being muxed changes to match the new command encoding.
<code>ddr3mc_flow_control.v</code>	Master FSM: CFG → INIT → CAL → NORMAL, refresh interleaving, address decode	T3	The phase-sequencing shape is a solid template. Refresh management must change substantially—DDR5 uses same-bank refresh (REFsb), not simple per-rank refresh, so this needs new states, not just new constants.
<code>ddr3mc_cfg.v</code>	Configuration register file: timing parameters, DIMM geometry, mode register values	T3	Register-file pattern reusable; field set must be redefined – DDR5 has roughly 40+ mode registers versus DDR3's 4

File	Role in DDR3 design	Tier	Notes for DDR5
			(MR0-MR3), and several new timing parameters (e.g. same-bank refresh timing) have no DDR3 equivalent.
<code>ddr3mc_init_engine.v</code>	DDR3 power-up sequence: reset, CKE assertion, MR0-MR3 programming, ZQCL	T4	DDR5 power-up uses a different power-state signaling scheme (no CKE in the DDR3 sense) and a much larger mode register set. The state-machine <i>shape</i> (wait → assert → program → calibrate) is worth referencing, but content is not portable.
<code>ddr3mc_rw_control.v</code>	Generates ACT/READ/WRITE/PRE/REF command sequences via discrete RAS _n /CAS _n /WE _n	T4	This is the most important rewrite in the project. DDR5 commands are encoded on a multiplexed <code>CA[13:0]</code> bus across two unit intervals — there is no RAS/CAS/WE distinction to drive. The turnaround-timing state machine (tCCD/tWTR-equivalent enforcement) is a useful design reference, but the signal generation

File	Role in DDR3 design	Tier	Notes for DDR5
			must be built from scratch.
ddr3mc_processing_engine.v	Selects per-rank rw_control instance, manages WDQ/RDQ data movement, burst timing	T3	The dispatch/data-movement pattern is reusable. Two structural changes are needed: burst counting logic for BL16 instead of BL8, and likely duplication into two parallel instances per DIMM since DDR5 splits each DIMM into independent 32-bit sub-channels rather than one 64-bit channel.
ddr3mc_datapath.v	Width conversion between system bus and PHY data path	T3	Conversion logic concept reusable; the width ratios change with BL16 and sub-channel splitting.

3. Calibration subsystem

DDR5 still needs DQ/DQS training – the eye gets narrower at higher data rates, so this is *more* important, not less. Most of the per-step training concepts transfer; the orchestration pattern is good. The new requirement is training the CA bus itself, which DDR3 never had to do.

File	Role in DDR3 design	Tier	Notes for DDR5
ddr3mc_cal_engine.v	Top-level calibration orchestrator, instantiates all steps	T3	Orchestrator pattern reusable; needs an additional step added for CA bus training (see Section 8).
ddr3mc_cal_flow_control.v	Sequences the 6 calibration steps with enable/done handshake	T3	Handshake pattern reusable directly; step list needs reordering/expansion.
ddr3mc_cal_preamble_det.v + _wrapper.v	Detects DQS preamble per lane	T2	DQS preamble detection concept is unchanged in DDR5; port with lane-count/width updates.
ddr3mc_cal_clock_centering.v + _wrapper.v	Centers read DQS in the data eye	T2	Same concept applies; DDR5's narrower eye at higher speed makes this calibration tighter but not different in kind.
ddr3mc_cal_dq_detect.v + _wrapper.v	Per-bit DQ delay alignment	T2	Same concept applies, arguably more critical at DDR5 speeds.
ddr3mc_cal_write_leveling.v + _wrapper.v	Write leveling via MR1 feedback bit, sweeping DQS odelay	T3	Fly-by topology compensation is still required in DDR5, but the trigger mechanism (which mode register bit, what feedback path) changes — verify against the DDR5 spec before reusing the sweep logic.
ddr3mc_cal_write_dqs_centering.v + _wrapper.v	Centers write DQS in the	T2	Same concept applies.

File	Role in DDR3 design	Tier	Notes for DDR5
write data eye			
ddr3mc_cal_read_write_sanity_check.v + _wrapper.v	Final pattern-based read/write verification	T2	Fully generic verification concept; only burst width/pattern length changes.
ddr3mc_cal_imux_pipe.v / ddr3mc_cal_omux_pipe.v	Routes active calibration step's data/commands	T3	Pipe-routing pattern reusable; needs adaptation to the new command bus signal set.
ddr3mc_iodelay_control.v	Translates calibration delay values into PHY delay-cell control sequences	T2	Reusable as-is if the new PHY uses the same delay-cell control protocol (ce/inc/load/cntvaluein). Verify against whatever delay cell variant is selected for DDR5 speed grades before assuming this ports unchanged.

4. Physical layer (PHY) and custom silicon cells

The PHY structure (byte lanes built from DQ/DQS/DM bit lanes, each with delay elements and DDR registers) is a sound architectural pattern that DDR5 PHYs still follow conceptually. The command/address driver does not survive, because DDR5 removed the discrete address pins it was built to drive.

File	Role in DDR3 design	Tier	Notes for DD
ddr3mc_phy_top.v	Aggregates byte lanes, control lanes, clock generators, PVT cal	T3	Aggregation pattern reuses composition changes — control lanes replaced by a CA-bus drive possibly duplicated per sub-channel.
ddr3mc_phy_byte_lane.v	One byte: 8× DQ bit lane + DQS bit lane + DM bit lane	T3	Byte-lane grouping holds. DDR5's data mask/DB handling differs from DDR3's simple DM—needs spec review (see D note below).
ddr3mc_phy_dq_bit_lane.v	One DQ bit: IOBUF + IDELAY/ODELAY + IDDRE1/ODDRE1	T3	The bidirectional-plus-delay-element pattern is the standard approach for DDR generation. At DDR5 data rates this likely needs a higher resolution delay cell and may need wider SERDES ratio (1:8 or 1:16) rather than a simple edge-p

File	Role in DDR3 design	Tier	Notes for DD
			register, given BL16 bursts.
ddr3mc_phy_dqs_bit_lane.v	DQS strobe bit lane	T3	DQS still exist DDR5; the bit-lane structure reusable, grouping ratio change with channel width
ddr3mc_phy_dm_bit_lane.v	Data mask output bit lane	T4	DDR5 change data masking/DBI semantics relative to DD — requires a fresh look at the spec rather than direct ports of this file's logic
ddr3mc_phy_ck_generator.v	Differential CK0/CK1 clock generation	T3	Concept reusable; instance count and grouping change with channel architecture.
ddr3mc_phy_control_lanes.v	Drives discrete RAS_n/CAS_n/WE_n/BA/A pins	T4	Replaced entirely by a CA-bus driver in DDR Not a useful starting point
ddr3mc_phy_pvt_cal.v	ZQ-resistor-based impedance calibration	T3	The ZQ calibration concept exists in DDR5 too; very exact procedure/timing

File	Role in DDR3 design	Tier	Notes for DD
			against the D spec, but the overall appro (sweep again external reference, converge via comparator) transfers.
IGIDDRT01A.v (custom DDR IO cell library)	Bidirectional/output/input ASIC IO cells	T3	Reusable as physical IP on the existing c library's spee grade suppor DDR5 data ra (4800+ MT/s DDR3's max ~2133 MT/s). Confirm with foundry/IP vendor (GUC whether a fas cell variant is needed.
DLL_DCC_v003.v , IGADLLT04A_MASTER/SLAVE/MATCH/PD_v003.v	DLL with duty-cycle correction, master/slave delay line cells	T3	The DLL-base delay calibration concept is generation-agnostic. Tap resolution an lock range wil likely need to increase for DDR5 timing margins — sa speed-grade caveat as the cell library.

File	Role in DDR3 design	Tier	Notes for DD
DELAY_S1.v DELAY_S4.v	Behavioral delay cell models	T3	Reusable pattern; verify tap count/resolution is adequate for DDR5 margin
iddre1.v oddre1.v	DDR input/output registers (capture/drive on both clock edges)	T3	The dual-edge capture concept is valid in DD but BL16 bursts may justify moving to a wider SERDE style deserial rather than a simple edge-p register — fla for architecture review rather than direct re
cal.v	PVT/ZQ calibration algorithm (pu/pd code sweep against comparators)	T3	Algorithm pattern reuse verify against DDR5 ZQ calibration time requirements

5. Serial Flash Controller (SFC) — boot flash loader

This entire subsystem manages a SPI NOR flash chip and has no relationship to the DRAM protocol generation – it is reusable essentially unchanged regardless of which DDR generation sits behind the memory controller.

File	Role	Tier
sfc_top.v, sfc_spim.v, sfc_spim_ctrl.v sfc_spim_read_control.v, sfc_spim_write_control.v, sfc_spim_erase_control.v, sfc_spim_top.v sfc_cmd_generator.v, sfc_io.v	SPI flash protocol implementation (read/write/erase)	T1
sfc_ddr3mc_transporter.v	Moves boot data from flash into DRAM	T2

6. SPD Decoder Controller (SPDC) — DIMM auto-configuration

This is the second major rewrite area, alongside the command bus. DDR5 replaced the simple SPD EEPROM-at-fixed-I2C-address model with an "SPD Hub" that also houses a PMIC and thermal sensor behind one addressable hub device, using a different byte layout (JEDEC Annex L) entirely.

File	Role in DDR3 design	Tier	Notes for DDR5
<code>spdc_top.v</code>	Top FSM managing the I2C read sequence	T4	The addressing model changes from a flat EEPROM read to hub-mediated access; the flow needs a redesign, not a port.
<code>spdc_i2c_clk.v</code> <code>spdc_i2c_st.v</code> <code>spdc_i2c_top.v</code>	Low-level I2C master (clock generation, START/ADDR/DATA/ACK/STOP state machine)	T2	If DDR5 SPD access stays on I2C, the bit-level protocol engine is reusable directly. Confirm whether the target hub uses I2C or I3C – if I3C, this needs replacing.
<code>spdc_dec.v</code> , <code>spdc_dec_beta.v</code>	Decodes raw SPD bytes into timing parameters	T4	Byte map is completely different under DDR5's SPD Annex L. Full rewrite required; nothing in the existing byte-offset table is reusable.
<code>spdc_pipeline.v</code> , <code>spdc_ctrl.v</code>	Pipeline registers and flow control around decode	T3	Control-flow pattern is reusable; rewire to the new decode logic.
<code>spdc_qadd.v</code> , <code>spdc_qdiv.v</code> , <code>spdc_qmults.v</code>	Q-format fixed-point arithmetic for MTB/FTB-to-clock-cycle conversion	T1	Generic fixed-point math utilities, independent of what they're converting.

7. Simulation and verification environment

The DRAM behavioral model itself cannot be reused (it is a DDR3-specific simulation model), but the surrounding testbench architecture and verification task patterns are a solid starting template.

File	Role	Tier	Notes for DDR5
ddr3.sv , ddr3_sim_sodimm_x8_2R.sv	Full DDR3 DRAM/DIMM behavioral model	T5	Needs a brand-new DDR5 SDRAM behavioral model – there is no incremental path from a DDR3 timing/command model.
spd_eeprom.sv 24AA02.v	SPD EEPROM behavioral model	T4	Needs replacement with an SPD Hub model matching DDR5's addressing scheme.
ddr3mc_tb_top.sv , tb_ddr3mc_top.sv	Top-level testbench: DUT + DRAM model + clock/reset + stimulus	T3	Testbench skeleton (instantiate DUT, generate clock/reset, wait for status bits, run stimulus) is a good template; the DUT and memory model underneath change entirely.
ddr3mc_sim_bfm.sv	Bus functional model with mem_write() / mem_read() tasks	T2	Task-based BFM pattern is reusable directly; task bodies need updating for any new command bus fields.
tb_phy_top.sv	PHY-only isolation testbench	T3	Useful template for isolating and validating the new PHY independent of the full controller.
4096Mb_ddr3_parameters.svh , 8192Mb_ddr3_parameters.svh	JEDEC DDR3 timing constants for the sim model	T5	Replace with DDR5 density/timing

File	Role	Tier	Notes for DDR5
			parameter files – no overlap in values.

8. New components required for DDR5—no DDR3 analog

These have no corresponding file in the DDR3 project and need to be designed from a blank sheet, informed by the DDR5 JEDEC specification:

- **CA bus command encoder/training engine** – DDR5 commands are encoded across two unit intervals on a multiplexed `CA[13:0]` bus rather than decoded from discrete RAS/CAS/WE pins. This replaces `ddr3mc_phy_control_lanes.v` and the command-generation half of `ddr3mc_rw_control.v`.
- **Command/address bus calibration step** – DDR3 only calibrated the DQ/DQS data path because address/command signals were low-speed enough not to need it. DDR5's CA bus runs fast enough to require its own training step, with no equivalent step in the existing `ddr3mc_cal_*` set.
- **Same-bank refresh (REFsb) management** – replaces the simple per-rank refresh counter in `ddr3mc_flow_control.v` with a more granular per-bank-group refresh scheme.
- **SPD Hub protocol handler** – manages the combined SPD/PMIC/thermal-sensor hub device, which has no equivalent in the SPDC subsystem.
- **Decision Feedback Equalization (DFE) control**, if the chosen DDR5 IO cells implement it – DDR3 signal rates didn't require receiver equalization; this is a new analog/digital control surface with no DDR3 precedent.
- **On-die ECC awareness in the data path**, if targeting DDR5 modules with on-die ECC – affects how the controller interprets read latency and error reporting, with nothing equivalent in the current `ddr3mc_processing_engine.v`.

9. Recommended porting sequence

1. **Start with Tier 1 items** (system bus, arbiter, queues, FIFOs, scrambler, SFC, fixed-point math) – these can be pulled into the new repository immediately with no design discussion needed.
2. **Lock the DDR5 sub-channel architecture decision early** – whether the processing engine and PHY get duplicated per 32-bit sub-channel affects nearly every Tier 3

module's port, so this should be settled before touching `ddr3mc_processing_engine.v` or `ddr3mc_phy_top.v`.

3. **Design the CA bus command encoder before touching calibration** – the calibration step sequence and `ddr3mc_rw_control.v` replacement both depend on how commands are represented internally once the CA bus encoder exists.
4. **Confirm PHYIO cell and DLL speed grade with the foundry/IP vendor before assuming any T3-tagged PHY file is a usable template** – if the existing GUC cell library doesn't support DDR5 data rates, the PHY byte-lane structure is still a useful pattern, but the underlying cells need to come from a different library generation.
5. **Treat SPDC and the simulation DRAM model as parallel, independent rewrite tracks** – neither blocks the controller-core work and both can be started by a separate engineer once the DDR5 SPD Annex L spec and a reference DDR5 behavioral model (vendor-provided or internally built) are available.

Appendix — Key DDR5 protocol differences driving these reuse decisions

DDR3 characteristic	DDR5 characteristic	Files most affected
Discrete RAS_n/CAS_n/WE_n/BA/A command pins	Multiplexed CA[13:0] bus, command encoded over 2 unit intervals	<code>ddr3mc_rw_control.v</code> , <code>ddr3mc_phy_control_lanes.v</code>
Burst length 8 (BL8)	Burst length 16 (BL16), BL32 in some modes	<code>ddr3mc_processing_engine.v</code> , <code>ddr3mc_queues.v</code> , <code>ddr3mc_datapath.v</code> , IO bit lanes
Single 64-bit channel per DIMM	Two independent 32-bit sub-channels per DIMM	<code>ddr3mc_processing_engine.v</code> , <code>ddr3mc_phy_top.v</code>
Mode registers MR0-MR3 (4 total)	40+ mode registers	<code>ddr3mc_init_engine.v</code> , <code>ddr3mc_cfg.v</code>
Per-rank refresh	Same-bank refresh (REFsb)	<code>ddr3mc_flow_control.v</code>
SPD: flat EEPROM at fixed I2C address	SPD Hub: combined SPD/PMIC/thermal sensor, JEDEC Annex L byte layout	All <code>spdc_*.v</code> files

DDR3 characteristic	DDR5 characteristic	Files most affected
Max data rate ~2133 MT/s	4800+ MT/s	PHY IO cells, DLL cells, delay elements
No CA bus training needed	CA bus requires its own signal-integrity training	New calibration step (Section 8)
DQ/DQS-only calibration	DQ/DQS calibration plus possible DFE control	<code>ddr3mc_cal_*.v</code> files, PHY bit lanes